arXiv:2602.05863v4 [cs.LG] 2 Sep 2026

Constrained Group Relative Policy Optimization

Roger Girgis[*]
Mila – Quebec AI Institute
École Polytechnique de Montréal

Rodrigue de Schaetzen[*]
Mila – Quebec AI Institute
Université de Montréal

Luke Rowe
Mila – Quebec AI Institute
Université de Montréal

Azalée Robitaille
Mila – Quebec AI Institute
Université de Montréal

Christopher Pal
Mila – Quebec AI Institute
École Polytechnique de Montréal
Université de Montréal, CIFAR AI Chair

Liam Paull
Mila – Quebec AI Institute
Université de Montréal, CIFAR AI Chair

Abstract

Group Relative Policy Optimization (GRPO) remains the dominant critic-free approach for fine-tuning LLMs and VLMs, but its compatibility with constrained policy optimization (e.g. for safety-critical domains) has not been carefully examined. In this work, we introduce Constrained GRPO, a Lagrangian-based extension of GRPO for constrained policy optimization. We show that the standard practice of scalarizing rewards before normalization introduces a critical Lagrangian-specific failure mode: GRPO's within-group normalization makes constrained optimization highly sensitive to how multi-component learning signals are aggregated. We show that scalarizing rewards before normalization introduces shared-denominator coupling, so that changing one multiplier alters not only the emphasis on its corresponding constraint, but also the relative weighting of the reward and other constraints. We address this with a simple but crucial modification: scalarizing standardized advantages rather than rewards. This yields a better-conditioned update by addressing the coupling induced by reward scalarization, resulting in better-behaved multiplier dynamics and more stable constraint enforcement in practice. Empirically, across a controlled gridworld, a real-world autonomous driving benchmark, and a mathematical reasoning task, Constrained GRPO consistently achieves better adherence to specified constraints while maintaining or improving task performance.

1 Introduction

Recent progress in reinforcement learning (RL) has shown that large multimodal models can be optimized with outcome-based signals. This has enabled the finetuning of models on mathematical reasoning tasks [1–3], embodied decision making [4], among other domains where success can be evaluated with programmatic or learned rewards. Group Relative Policy Optimization (GRPO) [5] removes the reliance on a critic for advantage estimation by a within-group normalization of the rewards obtained over multiple sampled outputs for the same query. The critic-free structure of GRPO is appealing when using a value function during training is challenging / computationally expensive.

[*]Equal contribution. Primary contact: roger.girgis@gmail.com

Preprint.

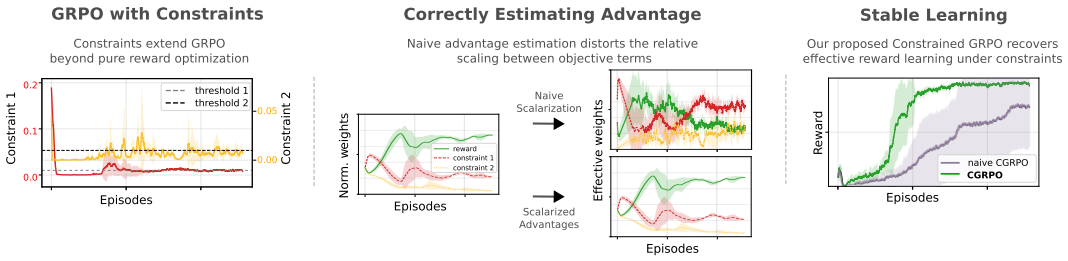

Figure 1: Overview of our proposed approach. We extend GRPO to constrained policy optimization by enforcing user-specified behavioral constraints with thresholds (left). Naively scalarizing reward distorts reward–constraint trade-offs, producing inconsistent effective weights (middle; e.g., red/green swap). Scalarized advantages maintains the relative multiplier weighting between advantage components, yielding more stable learning and effective constraint enforcement (right).

Existing GRPO-based methods have largely approached multi-objective optimization through manually weighting the importance of different reward terms [6–8]. In contrast, the constrained policy optimization literature offers a principled framework, namely Constrained Markov Decision Processes (CMDPs), for enforcing behavioural preferences and constraints by explicitly defining thresholds on behaviours rather than employing fixed weights. However, GRPO has not yet been integrated into a constrained optimization framework. In this work, we propose *Constrained GRPO*, a constrained policy optimization framework employing GRPO as the policy optimization algorithm. Within our framework, we specify desirable behaviours with indicator cost functions [9] and enforce target behaviour rates (in percentage) using a Lagrangian relaxation with learned multipliers.

A central challenge arises from the interaction between Lagrangian methods and GRPO's advantage estimation through within-group standardization. A simple approach is to first *scalarize rewards*, i.e., combine reward and cost terms into a single scalar value using the current multipliers, and then apply the within-group standardization. However, we show that this seemingly innocuous choice introduces an additional source of coupling in the constrained update: because the scalarized signal is normalized by a shared, multiplier-dependent denominator, changing one multiplier can affect not only its corresponding objective term, but also the relative contribution of of other components to the policy update. We provide a formal analysis of the gradient estimator sensitivity to changes in the multipliers, as well as an intuitive framing of how multiplier distortion occurs (e.g. Figure 1). Motivated by this analysis, Constrained GRPO employs *scalarizing advantages*, i.e., standardize each component within group first, and then combine the standardized components using the Lagrangian multipliers. This design yields better-conditioning and well-behaved multiplier dynamics by removing the shared-denominator coupling that is induced by reward scalarization. Importantly, this does not imply exact preservation of the original unnormalized Lagrangian objective; rather, it produces a more stable learning signal in the normalized component basis, which we validate empirically.

Prior and concurrent work has also found advantage scalarization beneficial in the realm of multi-objective GRPO [6, 7]; our contribution places this observation in the context of constrained optimization, showing why it is essential for effective Lagrangian constraint enforcement under GRPO. Our main contributions are: (1) We introduce Constrained GRPO, a constrained policy optimization framework built on GRPO, using indicator cost functions, Lagrangian relaxation and scalarizing advantages to effectively enforce behaviour rates without manually tuning reward weights. (2) We analyze the common approach of scalarizing rewards before normalizing, and formally characterize the local response of the primal update to changes in the dual variables in Theorem 4.1. (3) We study the alternative approach of scalarizing advantages and empirically demonstrate improved stability and faithfulness of the behaviour specification in a gridworld environment, a realistic simulation-based autonomous driving setting, and a mathematical reasoning task.

2 Related Work

Reward Specification. A central motivation for our work is the difficulty of specifying complex behaviours through a single reward function. This challenge has inspired several alternative approaches in the literature. Imitation Learning [10] bypasses reward engineering entirely by using expert demonstrations to define the task. Other methods introduce human feedback during training, either to guide the agent toward desired behaviours [11] or to prevent catastrophic errors during exploration [12]. Our approach shares a similar motivation—capturing designer intent more directly—but does so by

specifying constraints via indicator functions and thresholds, avoiding the need for demonstrations or continuous human intervention. Another promising direction is the use of natural language to specify tasks [13, 14]. While this can provide a more intuitive interface for human designers, language is inherently ambiguous and can be exploited by agents (reward hacking). Moreover, these methods typically require training an additional language-to-reward model, adding further complexity. Another interesting approach is Inverse Reward Design [15–17], which treats the provided reward function as a single observation of the designer's latent intent and learns a probabilistic model over possible true reward functions. This is particularly useful in settings with changing environments, but our work focuses on fixed-distribution environments where explicit behaviour specification is more appropriate.

Multi-Objective Optimization with Large Models. In the context of large multimodal (vision-) language models, there is extensive work on safety and multi-objective optimization via reinforcement learning [18–22]. For example, Constitutional AI [22] can be viewed as balancing multiple objectives—most notably helpfulness and harmlessness—yet in practice these objectives are typically reduced to a single scalar reward and optimized with standard RLHF [23] pipelines, for instance PPO-style updates [24]. Moskovitz et al. [18] frame composite reward models in RLHF as a constrained optimization problem, enforcing thresholds on individual reward components via a CMDP formulation and using dynamic weight adjustment (i.e., Lagrange multipliers) to mitigate over-optimization. Similarly, Dai et al. [20] formulate alignment as maximizing reward subject to cost constraints in RLHF, explicitly trading off helpfulness against safety-related costs. Finally, Wang et al. [25] study multi-objective reward models in RLHF, emphasizing interpretability as a means to help designers verify adherence to human preferences.

While several recent methods target safety alignment for RL-finetuned (vision-)language models, they are largely developed outside the GRPO setting. More recent work [6] train a multi-head reward model that predicts $K{=}4$ attributes (politeness, meaningfulness, actionability, safety) and then form a single scalar reward via a fixed weighted sum (using $w_k{=}1$ for all components), before optimizing the resulting objective with GRPO. Ichihara et al. [7] propose Multi-Objective GRPO (MO-GRPO) to reduce reward-hacking behavior under multiple objectives, and advocate scalarized advantages to equalize the influence of components with different variances. Concurrent to our work, GDPO [8] also supports advantage scalarization and emphasizes the difficulty of selecting reward weights that reflect desired priorities, proposing conditional objectives that activate certain terms only once prerequisite scores exceed a minimum threshold. In contrast, we study advantage scalarization through the lens of *constrained* policy optimization. Our goal is to ensure that increases in the Lagrangian multipliers in Constrained GRPO translate into stronger enforcement of the specified behaviors. As shown experimentally, scalarizing advantages provides this property, whereas scalarizing rewards can implicitly reweight objectives and undermine effective Lagrangian optimization.

3 Background

Lagrangian methods. Constrained Markov Decision Processes (CMDPs) are commonly addressed with Lagrangian methods [26–29, 4], which cast the problem as a saddle-point optimization [30–32]. These methods introduce K nonnegative multipliers $\boldsymbol{\lambda} = \{\lambda_k\}_{k=1}^{K}$, where each $\lambda_k \geq 0$ controls the penalty for the k-th constraint C_k. Letting $J_R(\pi)$ and $J_{C_k}(\pi)$ denote the expected returns of the reward R and the constraint C_k under policy π, with constraint threshold d_k, the resulting min–max problem is

$$\max_{\pi} \min_{\boldsymbol{\lambda} \geq 0} \mathcal{L}(\pi, \boldsymbol{\lambda}), \quad \text{where} \quad \mathcal{L}(\pi, \boldsymbol{\lambda}) = J_R(\pi) - \sum_{k=1}^{K} \lambda_k \big(J_{C_k}(\pi) - d_k\big). \tag{1}$$

A standard approach applies gradient ascent in π and gradient descent in $\boldsymbol{\lambda}$ [33], which often yields good feasible solutions on CMDP benchmarks [26–29].

Prior work [9] employed multiplier normalization:

$$\lambda_k \;=\; \frac{\exp(z_k)}{\exp(z_R) + \sum_{k'=1}^{K} \exp(z_{k'})}, \quad k = 1, \ldots, K. \tag{2}$$

which includes a dummy variable z_R for the main reward function such that $\lambda_R := 1 - \sum_{k=1}^{K} \lambda_k$ and z_k's are the *unnormalized* multipliers (logits). In this work, we adopt this mechanism since it ensures

that one constraint can dominate the policy updates if necessary while maintaining stability [9]. The policy update (i.e., primal update) now corresponds to optimizing a linear combination of the reward and constraint signals,

$$S^{\boldsymbol{\lambda}} = \lambda_R R - \sum_{k=1}^{K} \lambda_k C_k, \tag{3}$$

and can be carried out with any policy optimization method (e.g., PPO [24], SAC [34]). The multiplier update (i.e., dual update) follows

$$\nabla_{\lambda_k} \mathcal{L}(\pi, \boldsymbol{\lambda}) = -(J_{C_k}(\pi) - d_k), \tag{4}$$

typically implemented with a learning rate and a projection onto $\lambda_k \geq 0$ using max-clipping. If constraint k is violated ($J_{C_k} > d_k$), λ_k increases, strengthening its penalty; if it is satisfied ($J_{C_k} < d_k$), λ_k decreases, allowing optimization to focus on return and other active constraints.

Group-Relative Policy Optimization (GRPO). Reinforcement learning has proven effective for finetuning large models, complementing supervised finetuning (SFT): while SFT supports instruction and format adherence, RL finetuning often improves generalization, particularly in out-of-distribution regimes [35]. Early work used PPO with learned reward models for preference optimization [23]. Assuming access to a queryable scalar reward (e.g., BLEU/ROUGE, verifier-based math checking), Shao et al. [5] proposed eliminating the value model by estimating advantages within a group of samples drawn from the same prompt. Given a start state s_0, GRPO samples G candidate trajectories $\{\boldsymbol{\tau}_i = (s_0, a_0^i, \ldots, s_{T-1}^i, a_{T-1}^i, s_T^i)\}_{i=1}^G$, from the behavior policy $\boldsymbol{\tau}_i \sim \pi_{\theta_{\text{old}}}(\cdot \mid s_0)$, and computes group-relative advantages by standardizing returns:

$$A_i = \frac{R(\boldsymbol{\tau}_i) - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}, \quad \mathbf{r} = [R(\boldsymbol{\tau}_1), \ldots, R(\boldsymbol{\tau}_G)].$$

Using A_i, the PPO objective is modified into GRPO:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{\{\boldsymbol{\tau}_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{G} \sum_{i=1}^{G} \sum_{t=0}^{T-1} \{\min[b_1 \cdot A_i, b_2 \cdot A_i] - \beta \mathbb{D}_{\text{KL}}[\pi_\theta \mid\mid \pi_{\text{ref}}]\} \right],$$
$$\text{where} \quad b_1 = \frac{\pi_\theta(a_t^i \mid s_t^i)}{\pi_{\theta_{\text{old}}}(a_t^i \mid s_t^i)}, \quad b_2 = \text{clip}\left(\frac{\pi_\theta(a_t^i \mid s_t^i)}{\pi_{\theta_{\text{old}}}(a_t^i \mid s_t^i)}, 1+\epsilon, 1-\epsilon \right),$$

where the KL term (scaled by hyperparameter $\beta \geq 0$) directly regularizes the trained policy toward a fixed reference policy π_{ref}, rather than being folded into the reward. GRPO has shown strong performance in mathematical reasoning [5] and has been adopted in various robotics work [36–39].

4 Constrained GRPO

We study constrained policy optimization with GRPO as the policy-gradient algorithm and Lagrangian relaxation as the constraint-handling mechanism. Our analysis examines how the choice of advantage estimation in GRPO under multiple reward and constraint signals affects the primal-dual dynamics, and identifies the conditions under which the dual updates reliably translate into primal behavior. We assume one reward and K behavioral constraints; the extension to multiple rewards is straightforward.

In large-model finetuning, a common practice is to manually set the weights $\{\lambda_k\}$ and keep them fixed throughout training. We model each desired behavior k as a constraint and enforce it through a cost function C_k with a target threshold d_k and normalized multiplier λ_k (Eq. 2). A key design choice is how to express these thresholds in an intuitive and behavior-agnostic way. Following prior work on direct behavior specification [9], we restrict our attention to indicator cost functions $C_k(s,a) = \mathbb{I}\big[\text{constraint } k \text{ violated in } (s,a)\big]$, for which the threshold $\tilde{d}_k$ directly corresponds to a desired behavior rate. In this work, we focus on trajectory-level constraints and therefore drop the explicit dependence on (s,a).[2] We estimate constraint violations from a batch of experience by the empirical average of the cost C_k. The Lagrangian parameters are optimized using the update rule in Eq. (4), with $d_k \to \tilde{d}_k$ and $J_{C_k}(\pi)$ computed as the average over $\{C_k\}_{i=1}^N$.

[2]The analysis in this work easily extends to state-action-level constraints.

GRPO Advantage Estimation. GRPO estimates advantages by within-group standardization: subtracting the group mean and dividing by the group standard deviation. There are two natural ways to compute the advantage. The first scalarizes at the reward level by linearly combining rewards and costs with the Lagrange multipliers before estimating the advantage. The second scalarizes at the advantage level by first computing component-wise advantages for each reward and cost and then linearly combining them with the Lagrange multipliers. In what follows, we analyze the differences between these two approaches using a sensitivity analysis on the unclipped policy-gradient, revealing why scalarized advantages offers a more practical approach from the perspective of primal learning dynamics. Note that this sensitivity analysis is taken with respect to the logits z_k (see Eq. 2) rather than treating the multipliers λ_k as independent coordinates. This is a consequence of the multiplier normalization that we employ and we provide the corresponding sensitivity analysis for the unnormalized case in Appendix A.1 (up to Eq. 13).

Scalarized rewards. The most direct approach first forms a scalarized signal $S^{\boldsymbol{\lambda}}$ as done in (3), and then applies GRPO normalization:

$$A_i^{\mathrm{ScRew}} = \frac{S_i^{\boldsymbol{\lambda}} - \bar{S}^{\boldsymbol{\lambda}}}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}}, \quad \hat{\sigma}_{S^{\boldsymbol{\lambda}}} = \mathrm{std}(S_1^{\boldsymbol{\lambda}}, \dots, S_G^{\boldsymbol{\lambda}}).$$

Subscript i denotes the i-th sample in the group, and $\bar{\cdot}$ denotes the empirical mean over the group. Let

$$\psi_i := \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i),$$

and define

$$N_R := \frac{1}{G}\sum_{i=1}^{G}(R_i - \bar{R})\psi_i, \qquad N_k := \frac{1}{G}\sum_{i=1}^{G}(C_{k,i} - \bar{C}_k)\psi_i.$$

Then the scalarized-reward constrained GRPO estimator can be written as

$$g^{\mathrm{ScRew}}(\boldsymbol{\lambda}) = \frac{\lambda_R N_R - \sum_{k=1}^{K}\lambda_k N_k}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}}. \tag{5}$$

We now compute the multiplier sensitivities of g^{ScRew}:

Theorem 4.1 (Multiplier sensitivity under scalarized rewards)**.** *For a fixed sampled group, the sensitivity of the scalarized-reward GRPO estimator g^{ScRew} to the logit z_k of constraint k is*

$$\frac{\partial g^{\mathrm{ScRew}}}{\partial z_k} = \lambda_k \left(\frac{\partial g^{\mathrm{ScRew}}}{\partial \lambda_k} - \sum_{j \in \{R,1,\dots,K\}} \lambda_j \frac{\partial g^{\mathrm{ScRew}}}{\partial \lambda_j} \right), \tag{6}$$

where, letting $D := \lambda_R N_R - \sum_{k=1}^{K}\lambda_k N_k$, the partial sensitivities are

$$\frac{\partial g^{\mathrm{ScRew}}}{\partial \lambda_R} = \frac{N_R}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}} - \frac{D}{\hat{\sigma}^3_{S^{\boldsymbol{\lambda}}}}\widehat{\mathrm{Cov}}(S^{\boldsymbol{\lambda}}, R), \qquad \frac{\partial g^{\mathrm{ScRew}}}{\partial \lambda_k} = -\frac{N_k}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}} + \frac{D}{\hat{\sigma}^3_{S^{\boldsymbol{\lambda}}}}\widehat{\mathrm{Cov}}(S^{\boldsymbol{\lambda}}, C_k),$$

and $\widehat{\mathrm{Cov}}(S^{\boldsymbol{\lambda}}, X) := \frac{1}{G}\sum_{i=1}^{G}(S_i^{\boldsymbol{\lambda}} - \bar{S}^{\boldsymbol{\lambda}})(X_i - \bar{X})$ denotes the empirical group covariance.

Proof in Appendix A.1. Observe that scalarized rewards induces *two* forms of coupling. First, the softmax normalization means that increasing one logit decreases the effective weight on the other components by construction. Second, the shared denominator $\hat{\sigma}_{S^{\boldsymbol{\lambda}}}$ introduces additional covariance-mediated coupling through $\partial g^{\mathrm{ScRew}}/\partial \lambda_R$ and $\partial g^{\mathrm{ScRew}}/\partial \lambda_k$. Consequently, changing one logit affects not only the corresponding constraint term, but also the relative contribution of the reward and all other constraints, while simultaneously modifying the normalization of the mixed signal; the resulting primal update is therefore altered both in magnitude and in direction.

Scalarized advantages. Our method instead normalizes each component separately and then combines the resulting standardized advantages:

$$Z_{R,i} = \frac{R_i - \bar{R}}{\hat{\sigma}_R}, \qquad Z_{C_k,i} = \frac{C_{k,i} - \bar{C}_k}{\hat{\sigma}_{C_k}}.$$

The scalarized advantage is

$$A_i^{\mathrm{ScAdv}} = \lambda_R Z_{R,i} - \sum_{k=1}^{K} \lambda_k Z_{C_k,i}. \tag{7}$$

The corresponding estimator is

$$g^{\mathrm{ScAdv}}(\boldsymbol{\lambda}) = \lambda_R \frac{N_R}{\hat{\sigma}_R} - \sum_{k=1}^{K} \lambda_k \frac{N_k}{\hat{\sigma}_{C_k}}. \tag{8}$$

Corollary 4.2 (Multiplier sensitivity under scalarized advantages)**.** *For a fixed sampled group, the sensitivity of the scalarized-advantage GRPO estimator to the logit z_k of constraint k is*

$$\frac{\partial g^{\mathrm{ScAdv}}}{\partial z_k} = \lambda_k \left(-\frac{N_k}{\hat{\sigma}_{C_k}} - \lambda_R \frac{N_R}{\hat{\sigma}_R} + \sum_{k'=1}^{K} \lambda_{k'} \frac{N_{k'}}{\hat{\sigma}_{C_{k'}}} \right). \tag{9}$$

Unlike scalarized rewards, scalarized advantages does not introduce any additional shared-denominator coupling through the within-group normalization. With softmax-normalized multipliers, some competition between components is unavoidable, since increasing one logit reallocates mass away from the others. However, under scalarized advantages this is the *only* source of coupling: the logit update is a competition over already-normalized component directions. In contrast, under scalarized rewards the same simplex competition is compounded by the dependence of $\hat{\sigma}_S$ on the current reward-cost mixture, which adds covariance-mediated cross-coupling to the primal response. For these reasons, we adopt scalarized advantages in constrained GRPO.

With scalarized advantages, the effective multipliers being applied to the advantage components is λ_k by construction (i.e., Eq. 7). Whereas, Theorem A.3 shows that when scalarizing rewards, the multipliers are rescaled by $\tilde{\lambda}_k = \lambda_k \frac{\sigma_{C_k}}{\sigma_{S^{\boldsymbol{\lambda}}}}$ (likewise $\tilde{\lambda}_R$) such that $A_i^{\mathrm{ScRew}} = \tilde{\lambda}_R Z_{R,i} - \sum_{k=1}^{K} \tilde{\lambda}_k Z_{C_k,i}$.

5 Experiments

5.1 Illustrative Environment

We begin with an illustrative example in a simple gridworld. An agent is randomly initialized in a 10×10 grid and must navigate to a randomly sampled goal location, receiving a sparse scalar reward upon reaching it. Each episode contains 20% lava tiles; stepping onto lava incurs a penalty (i.e., a cost). In addition, the agent has a battery that depletes at every timestep and incurs a penalty whenever its battery level falls below 10%. The action space consists of five discrete actions: moving in the four cardinal directions or staying in place to recharge. This simple environment provides a controlled testbed for isolating the differences between *scalarized rewards* and *scalarized advantages* in constrained GRPO. Additional setup details and more experiments are provided in Appendix C.

We enforce constraint thresholds $\tilde{d}_{\text{lava}} = \tilde{d}_{\text{battery}} = 0.01$ and train for 512k episodes. Figure 2 compares Constrained GRPO with scalarized advantages (ours) against a scalarized rewards baseline. We see that scalarized advantages yields more stable learning across seeds (5 seeds), as reflected by the narrower shaded region throughout training (column 1). Moreover, scalarized advantages leads to richer behavior-rate dynamics (column 2, bottom row): the Lagrangian multipliers (column 3, bottom row) adapt to allow the constraint rates to increase toward the threshold, effectively encouraging the policy to use its available cost budget. On the other hand, scalarized rewards rapidly drives both constraint rates close to zero early in training (column 2, top row), and keeps them near-zero thereafter, even as the learned multipliers (columb 3, top row) remain small.

This behavior can be understood through the implicit reweighting predicted by Theorem A.3. Observe that despite the small learned multipliers λ_{lava} and λ_{battery}, the induced effective weights are larger and significantly noisier due to within-group variance and covariance effects (row 4, left column). This is especially problematic when the ordering of importance changes, as observed around 150K steps for the goal and lava weights (red/green swap). Overall, these results support *scalarized advantages* as a more reliable advantage construction for Constrained GRPO, as it preserves the intended role of the Lagrangian multipliers in enforcing behavior specifications.

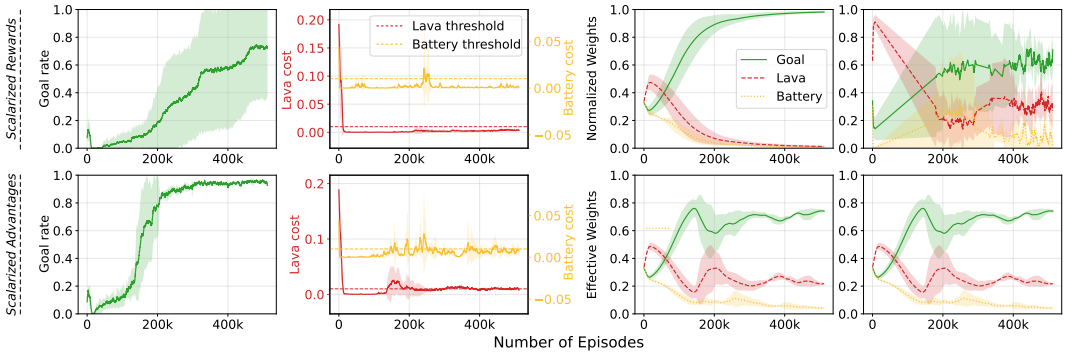

Figure 2: Gridworld under Constrained GRPO, comparing *scalarized rewards* (top) with *scalarized advantages* (bottom). Scalarized advantages yields more stable training and uses the available cost budget, with behavior rates adapting toward the specified thresholds as the Lagrangian multipliers adjust. In contrast, scalarized rewards collapses constraint rates toward near-zero early in training with the effective weights remaining larger and noisier despite small learned multipliers.

5.2 NAVSIM Experiments

Benchmark Details.

Our next experiments are on the NAVSIM-v2 driving benchmark [40, 41], which contains 103k training scenarios (Navtrain) and 12k testing scenarios (Navtest) curated from OpenScene [42, 43]. NAVSIM-v2 evaluates planning using explicit behavioural safety constraints alongside task-level objectives such as progress and comfort, enabling targeted analysis of behaviour specification. We do our evaluations on the *Navhard* split containing more challenging driving scenarios and planners are evaluated using a two-stage pseudo-simulation protocol: Stage 1 evaluates the policy's ability to drive in unseen nominal driving scenarios while stage 2 evaluates the policy's ability to produce corrective behaviors. More details are in Appendix D.

The primary evaluation metric is Extended PDM Score (EPDMS $\in [0, 1]$) which aggregates multiple subscores via a product of hard penalty terms and a weighted average of auxiliary terms (Eq. 22 in Appendix D). The penalty terms, each taking values in $\{0, 1\}$, include: (1) no at-fault collisions (NC), (2) drivable area compliance (DAC), (3) driving direction compliance (DDC), and (4) traffic light compliance (TLC). Since these penalties are combined multiplicatively, violating any one of them sets EPDMS$= 0$ for that scene. The weighted average terms consist of: (1) ego progress (EP) along the route, (2) time-to-collision (TTC), (3) history comfort (HC), (4) extended comfort (EC), and (5) lane keeping (LK). We also report the 4-second Average Displacement Error (ADE@4s) on Stage 1 scenes, computed with respect to the human trajectory.

Although the penalty terms are designed as hard constraints at evaluation time, directly optimizing a multiplicative hard-gated objective can lead to sparse learning signals and unstable optimization, as we show. We therefore treat penalty terms as soft behavioural constraints, targeting near-perfect satisfaction (0.99) while preserving informative gradients throughout training.

Model Baselines. We initialize all experiments with the Qwen2.5-VL 3B Instruct model [44], which we pre-train following the Poutine supervised finetuning (SFT) recipe [45]. We then perform additional SFT on the NAVSIM training set [40, 41] (see Appendix D), reserving $\approx$ 11k scenes as a held-out split for RL finetuning (RLFT). Our baselines are as follows:[3]

(1) *GRPO with EPDMS*: optimize EPDMS as the reward, in the spirit of CaRL [46] (which optimizes a similar objective with PPO). Because EPDMS is multiplicative, any NC/DAC/DDC/TLC violation collapses the reward to zero, acting as a hard constraint.

(2–5) *Linear scalarization:* a weighted sum of constraints and the weighted-average terms (WAT): $R(\tau) = w_1\text{NC}+w_2\text{DAC}+w_3\text{DDC}+w_4\text{WAT}$. This follows prior multi-objective GRPO setups [6, 7]. We evaluate both scalarization choices: scalarize rewards (*ScRew*) and scalarize advantages (*ScAdv*) and consider two weighting schemes: (i) equal weights $w_i = 0.25$ (*GRPO, ScRew/ScAdv, Equal Weights*), and (ii) informed weights tuned from SFT behavior w_1=0.1, w_2=0.8, w_3=0.05, w_4=0.05 (*GRPO, ScRew/ScAdv, Informed Weights*).

[3]Traffic light compliance (TLC) was almost always satisfied in preliminary experiments and is therefore omitted as an explicit objective, except when optimizing EPDMS.

Method	Constraints	Stage	NC ↑	DAC ↑	DDC ↑	TLC ↑	EP ↑	HC ↑	EPDMS ↑	ADE@4s ↓
PDM-Closed (privileged) [47]	–	S1	0.944	0.988	1.000	0.995	1.000	0.877	0.834	–
		S2	0.881	0.906	0.963	0.985	1.000	0.915	0.687	–
SFT Checkpoint	–	S1	0.953	0.789	0.978	0.993	0.836	0.976	0.651	**2.539**
		S2	0.800	0.756	0.869	0.981	0.878	0.963	0.444	–
GRPO with EPDMS [46]	Hard	S1	0.897	0.800	0.947	0.990	0.944	**0.978**	0.612	6.900
		S2	0.749	0.759	0.829	0.969	0.979	0.945	0.381	–
GRPO, ScRew, Equal Weights [6]	None	S1	0.891	0.625	0.876	0.989	**0.968**	0.976	0.445	9.050
		S2	0.751	0.696	0.787	0.968	**0.986**	0.954	0.347	–
GRPO, ScAdv, Equal Weights [7]	None	S1	0.908	0.661	0.900	0.989	0.954	0.973	0.490	7.172
		S2	0.767	0.705	0.813	0.971	0.981	0.949	0.370	–
GRPO, ScRew, Informed Weights	None	S1	0.887	0.643	0.887	0.987	**0.967**	**0.978**	0.463	8.509
		S2	0.754	0.695	0.791	0.969	**0.986**	0.954	0.350	–
GRPO, ScAdv, Informed Weights	None	S1	0.932	0.765	0.971	0.993	0.890	0.976	0.623	3.495
		S2	0.778	0.731	0.831	0.972	0.944	0.962	0.385	–
PPO with EPDMS	Hard	S1	0.944	0.731	0.945	0.991	0.862	0.976	0.589	4.057
		S2	0.794	0.727	0.855	0.969	0.908	0.963	0.415	–
PPO-Lagrangian [27]	Soft	S1	**0.986**	0.891	**0.994**	**0.998**	0.721	**0.978**	0.737	4.558
		S2	**0.895**	0.812	**0.912**	**0.990**	0.767	**0.980**	**0.536**	–
Constrained GRPO, ScRew	Soft	S1	0.955	0.887	0.987	0.993	0.868	0.976	0.743	3.344
		S2	0.810	0.784	0.866	0.976	0.939	0.952	0.436	–
Constrained GRPO, ScAdv (ours)	Soft	S1	0.957	**0.928**	0.989	**0.998**	0.796	**0.977**	**0.774**	3.361
		S2	0.853	**0.830**	0.897	0.986	0.863	0.967	0.516	–

Table 1: **Results on NAVSIM-v2 Navhard.** We compare several baselines with Constrained GRPO in Stage 1 (S1, real scenes) and Stage 2 (S2, synthetic scenes). The Constraints column indicates whether NC/DAC/DDC/TLC are treated as hard constraints, soft constraints, or not enforced. *ScRew*: Scalarized Rewards; *ScAdv*: Scalarized Advantages.

(6–7) *Constrained GRPO:* treat WAT is as the main objective and enforce NC/DAC/DDC as constraints, setting all constraint satisfaction rate thresholds to 0.99 [4]. We again compare scalarized rewards and scalarized advantages (*Constrained GRPO, ScRew* vs. *Constrained GRPO, ScAdv*).

(8) We also compare against *PPO and Constrained PPO*. The former uses EPDMS as the reward while the latter uses the standard Lagrangian variant of PPO [27, 28] where the constraints are identical to Constrained GRPO described above.

Results. Tables 1 reports results on NAVSIM-v2 Stage 1 and Stage 2, respectively. For context, we also include the performance of the rule-based PDM planner [47], whose trajectories are used to compute EP. We now summarize the key findings:

Behavior specification through constrained RL outperforms alternative strategies. Our results indicate that constrained RL provides a more effective mechanism for behavior specification than alternative reward-design approaches. In particular, the two constrained GRPO variants (bottom two rows of Table 1) achieve strong overall performance, with constrained GRPO using scalarized advantages attaining the highest S1 EPDMS and second highest S2 EPDMS. On in-distribution Stage 1 driving scenarios, both constrained variants substantially improve upon the SFT checkpoint, whereas the unconstrained baselines do not consistently yield gains. On the more challenging out-of-distribution Stage 2 split, all methods exhibit degraded performance relative to S1 scores, as expected. Nevertheless, our approach remains the most competitive among GRPO baselines and achieves comparable performance with PPO-Langagrian, showcasing the usefulness of our behavior specification framework for limiting bad driving behaviors.

Scalarized advantages outperforms scalarizing rewards. Across all three linear formulations (equal weights, informed weights, and constrained), scalarizing advantages yields consistently higher EPDMS than scalarizing rewards, for both Stage 1 and Stage 2. This trend aligns with our analysis in Section 4: when objective components have mismatched scales, scalarizing rewards can implicitly bias the optimization toward high-variance terms, distorting the intended trade-offs.

We further illustrate this effect in Figure 6 in Appendix D. Under constrained GRPO with ScRew, the learned Lagrangian multipliers do not translate into effective constraint enforcement. This is most evident for DAC and EP, which is a proxy for the WAT objective: despite DAC being assigned nearly the maximum multiplier weight (and the main reward weight approaching zero), DAC does not reach the target satisfaction level, while EP continues to increase and only slowly decays later in training. In contrast, constrained GRPO with ScAdv responds rapidly to multiplier updates. DAC improves within

[4] Note that to maintain the upper-bound form of the constraint, we instead enforce a $\tilde{d}_k = 0.01$ threshold on the negation of the penalty terms NC/DAC/DDC.

Method	# params	Length ↓	Format ↑	Integer ↑	Correctness ↑
PPO-Lagrangian	1.5B	346.93	0.886	0.935	0.604
GDPO [8]	1.5B	296.89	**0.953**	**0.981**	0.629
Constrained GRPO, ScRew	1.5B	**246.82**	0.934	0.941	0.640
Constrained GRPO, ScAdv (ours)	1.5B	293.36	**0.953**	0.966	**0.712**
PPO-Lagrangian	3B	753.10	0.759	0.867	0.642
GDPO [8]	3B	**330.11**	0.911	0.977	0.770
Constrained GRPO, ScRew	3B	334.61	0.965	0.968	0.751
Constrained GRPO, ScAdv (ours)	3B	349.93	**0.987**	**0.994**	**0.802**
PPO-Lagrangian	7B	327.74	0.980	0.995	**0.891**
GDPO [8]	7B	127.49	**0.990**	0.995	0.768
Constrained GRPO, ScRew	7B	**119.65**	0.988	0.993	0.731
Constrained GRPO, ScAdv (ours)	7B	256.62	0.989	**0.996**	0.887

Table 2: **GSM8K results across model scales.** We compare *PPO-Lagrangian*, *GDPO [8]*, and constrained GRPO with *scalarized rewards* and *scalarized advantages* with Qwen2.5-Instruct models at 1.5B, 3B, and 7B parameters. Constrained GRPO consistently improves correctness over GDPO at all scales while maintaining strong format and integer validity, and achieves competitive or better performance at 1.5B and 3B without requiring a learned critic (i.e., PPO-Lagrangian). Bold indicates the best; Underlined values indicate results that are close to the best.

the first $\sim$1000 training steps, while EP decreases as its multiplier is reduced, demonstrating that the Lagrangian weights exert their intended influence on the learning dynamics. Finally, around $\sim$2500 steps, the NC multiplier increases sharply as DAC becomes satisfactory but collision compliance degrades, showing that the optimizer adaptively reallocates emphasis across constraints.

Hard constraints lead to suboptimal learning dynamics. Finally, observe that directly optimizing EPDMS as a reward (i.e., GRPO with EPDMS) underperforms the soft-constraint formulations, yielding lower overall EPDMS on both Stage 1 and Stage 2. Intuitively, EPDMS couples constraints through a multiplicative structure, collapsing the learning signal whenever any penalty term is violated. This removes the ability to trade off and prioritize objectives smoothly during training, and prevents variance-aware balancing of component signals. In contrast, our soft-constraint formulation retains informative gradients even when constraints are partially violated, enabling more stable optimization and ultimately improved behavior specification.

5.3 GSM8K Experiments

Benchmark Details. We further evaluate our approach in a language reasoning setting using the GSM8K dataset, which consists of grade-school math word problems requiring multi-step reasoning. For each completion, we consider four signals: correctness, format adherence, integer validity, and length (which favors shorter reasoning chains).

Model Baselines. We initialize all experiments from Qwen2.5-Instruct models at three scales: 1.5B, 3B, and 7B parameters. For the baselines, we compare to *GDPO* and *PPO-Lagrangian*. GDPO [8] is a prior advantage-scalarization baseline designed for multi-objective GRPO training. PPO-Lagrangian uses a learned value function and enforces constraints through the standard primal-dual PPO formulation. For our constrained formulation, we treat correctness, format adherence, and integer validity as constraints, and optimize the length objective as the primary reward. Our goal is not to claim that correctness is only required up to a threshold; rather, the thresholds serve as optimization targets that prevent the policy from trivially improving length by sacrificing correctness or output validity.

Results. Tables 2 reports our GSM8K results. Across all three model scales, constrained GRPO achieves higher correctness than GDPO while maintaining strong performance on format adherence and integer validity. The gains are most pronounced at larger scale: at 3B, constrained GRPO improves correctness from 0.770 to 0.802, and at 7B from 0.768 to 0.887. These results suggest that explicitly enforcing correctness as a constraint prevents the model from over-optimizing easier auxiliary objectives such as reducing output length. We observe that GDPO often produces substantially shorter generations, especially at the 7B scale, where it reduces length to 127.49 tokens on average but at the expense of correctness. Compared to PPO-Lagrangian, constrained GRPO achieves comparable or better performance on all four metrics at the 1.5B and 3B scales. At 7B,

PPO-Lagrangian attains slightly higher correctness (0.891 vs. 0.887), but constrained GRPO remains competitive and produces shorter outputs without requiring expensive critic training.

6 Conclusion

We introduced Constrained GRPO, a Lagrangian extension of GRPO that enforces user-specified constraints via indicator costs and learned multipliers. We showed that scalarizing rewards before normalization introduces shared-denominator coupling, so that changing one multiplier can affect not only its corresponding constraint term, but also the relative contribution of the other terms to the policy update. By instead scalarizing standardized advantages, Constrained GRPO yields a better-conditioned update and more stable multiplier dynamics in practice.

7 Acknowledgements

We thank Samsung, the IVADO and the Canada First Research Excellence Fund (CFREF) / Apogee Funds, the Canada CIFAR AI Chairs Program, Fonds de Recherche Nature et Technologies (FRQNT), and the NSERC Discovery Grants program for their financial support. We thank Mila - Quebec AI Institute for compute resources. This research was supported by grants from NVIDIA and utilized NVIDIA 15K A100 GPU-hours on Saturn Cloud.

References

[1] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[2] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed H. Chi, F. Xia, Quoc Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. *ArXiv*, abs/2201.11903, 2022. URL `https://api.semanticscholar.org/CorpusID:246411621`.

[3] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Venkatesh Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models. *ArXiv*, abs/2206.14858, 2022. URL `https://api.semanticscholar.org/CorpusID:250144408`.

[4] Borong Zhang, Yuhao Zhang, Jiaming Ji, Yingshan Lei, Josef Dai, Yuanpei Chen, and Yaodong Yang. Safevla: Towards safety alignment of vision-language-action model via constrained learning. *arXiv preprint arXiv:2503.03480*, 2025.

[5] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Jun-Mei Song, Mingchuan Zhang, Y. K. Li, Yu Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *ArXiv*, abs/2402.03300, 2024. URL `https://api.semanticscholar.org/CorpusID:267412607`.

[6] Xuying Li, Zhuo Li, Yuji Kosuga, and Victor Bian. Optimizing safe and aligned language generation: A multi-objective grpo approach. *ArXiv*, abs/2503.21819, 2025. URL `https://api.semanticscholar.org/CorpusID:277435738`.

[7] Yuki Ichihara, Yuu Jinnai, Tetsuro Morimura, Mitsuki Sakamoto, Ryota Mitsuhashi, and Eiji Uchibe. Mo-grpo: Mitigating reward hacking of group relative policy optimization on multi-objective problems. *arXiv preprint arXiv:2509.22047*, 2025.

[8] Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Peter Belcak, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, et al. Gdpo: Group reward-decoupled normalization policy optimization for multi-reward rl optimization. *arXiv preprint arXiv:2601.05242*, 2026.

[9] Julien Roy, Roger Girgis, Joshua Romoff, Pierre-Luc Bacon, and Christopher Joseph Pal. Direct behavior specification via constrained reinforcement learning. In *International Conference on Machine Learning*, 2021. URL https://api.semanticscholar.org/CorpusID:245424738.

[10] Boyuan Zheng, Sunny Verma, Jianlong Zhou, Ivor Tsang, and Fang Chen. Imitation learning: Progress, taxonomies and opportunities. *arXiv preprint arXiv:2106.12177*, 2021.

[11] Paul Christiano, Jan Leike, Tom B Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *arXiv preprint arXiv:1706.03741*, 2017.

[12] William Saunders, Girish Sastry, Andreas Stuhlmueller, and Owain Evans. Trial without error: Towards safe reinforcement learning via human intervention. *arXiv preprint arXiv:1707.05173*, 2017.

[13] Prasoon Goyal, Scott Niekum, and Raymond J Mooney. Using natural language for reward shaping in reinforcement learning. *arXiv preprint arXiv:1903.02020*, 2019.

[14] James MacGlashan, Monica Babes-Vroman, Marie desJardins, Michael L Littman, Smaranda Muresan, Shawn Squire, Stefanie Tellex, Dilip Arumugam, and Lei Yang. Grounding english commands to reward functions. In *Robotics: Science and Systems*, 2015.

[15] Dylan Hadfield-Menell, Smitha Milli, Pieter Abbeel, Stuart Russell, and Anca Dragan. Inverse reward design. *arXiv preprint arXiv:1711.02827*, 2017.

[16] Sören Mindermann, Rohin Shah, Adam Gleave, and Dylan Hadfield-Menell. Active inverse reward design. *arXiv preprint arXiv:1809.03060*, 2018.

[17] Ellis Ratner, Dylan Hadfield-Menell, and Anca D Dragan. Simplifying reward design through divide-and-conquer. *arXiv preprint arXiv:1806.02501*, 2018.

[18] Ted Moskovitz, Aaditya K. Singh, DJ Strouse, Tuomas Sandholm, Ruslan Salakhutdinov, Anca D. Dragan, and Stephen Marcus McAleer. Confronting reward model overoptimization with constrained rlhf. *ArXiv*, abs/2310.04373, 2023. URL https://api.semanticscholar.org/CorpusID:263829192.

[19] Borong Zhang, Yuhao Zhang, Jiaming Ji, Yingshan Lei, Josef Dai, Yuanpei Chen, and Yaodong Yang. Safevla: Towards safety alignment of vision-language-action model via safe reinforcement learning. *ArXiv*, abs/2503.03480, 2025. URL https://api.semanticscholar.org/CorpusID:276782535.

[20] Josef Dai, Xuehai Pan, Ruiyang Sun, Jiaming Ji, Xinbo Xu, Mickel Liu, Yizhou Wang, and Yaodong Yang. Safe rlhf: Safe reinforcement learning from human feedback. *ArXiv*, abs/2310.12773, 2023. URL https://api.semanticscholar.org/CorpusID:264306078.

[21] Jiaming Ji, Xinyu Chen, Rui Pan, Han Zhu, Conghui Zhang, Jiahao Li, Donghai Hong, Boyuan Chen, Jiayi Zhou, Kaile Wang, Juntao Dai, Chi-Min Chan, Sirui Han, Yike Guo, and Yaodong Yang. Safe rlhf-v: Safe reinforcement learning from human feedback in multimodal large language models. *ArXiv*, abs/2503.17682, 2025. URL https://api.semanticscholar.org/CorpusID:277271998.

[22] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, John Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Chris Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, E Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, J Landau, Kamal Ndousse, Kamilė Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noem'i Mercado, Nova Dassarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, T. J. Henighan, Tristan Hume, Sam Bowman, Zac Hatfield-Dodds, Benjamin Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom B. Brown, and Jared Kaplan. Constitutional ai: Harmlessness from ai feedback. *ArXiv*, abs/2212.08073, 2022. URL https://api.semanticscholar.org/CorpusID:254823489.

[23] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022. URL https://arxiv.org/abs/2203.02155.

[24] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[25] Haoxiang Wang, Wei Xiong, Tengyang Xie, Han Zhao, and Tong Zhang. Interpretable preferences via multi-objective reward modeling and mixture-of-experts. In *Conference on Empirical Methods in Natural Language Processing*, 2024. URL https://api.semanticscholar.org/CorpusID:270562658.

[26] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *International Conference on Machine Learning*, pages 22–31. PMLR, 2017.

[27] Alex Ray, Joshua Achiam, and Dario Amodei. Benchmarking safe exploration in deep reinforcement learning. *arXiv preprint arXiv:1910.01708*, 7, 2019.

[28] Adam Stooke, Joshua Achiam, and Pieter Abbeel. Responsive safety in reinforcement learning by pid lagrangian methods. In *International Conference on Machine Learning*, pages 9133–9143. PMLR, 2020.

[29] Yiming Zhang, Quan Vuong, and Keith W Ross. First order constrained optimization in policy space. *arXiv preprint arXiv:2002.06506*, 2020.

[30] H Uzawa, K Anow, and L Hurwicz. Studies in linear and nonlinear programming, 1958.

[31] BT Polyak. Iterative methods using lagrange multipliers for solving extremal problems with constraints of the equation type. *USSR Computational Mathematics and Mathematical Physics*, 10(5):42–52, 1970.

[32] G.M. Korpelevich. An extragradient method for finding saddle points and for other problems. 1976.

[33] Tianyi Lin, Chi Jin, and Michael Jordan. On gradient descent ascent for nonconvex-concave minimax problems. In *International Conference on Machine Learning*, pages 6083–6093. PMLR, 2020.

[34] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.

[35] Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V. Le, Sergey Levine, and Yi Ma. Sft memorizes, rl generalizes: A comparative study of foundation model post-training. *ArXiv*, abs/2501.17161, 2025. URL https://api.semanticscholar.org/CorpusID:275932560.

[36] Yue Li, Meng Tian, Dechang Zhu, Jiangtong Zhu, Zhenyu Lin, Zhiwei Xiong, and Xinhai Zhao. Drive-r1: Bridging reasoning and planning in vlms for autonomous driving with reinforcement learning. *ArXiv*, abs/2506.18234, 2025. URL https://api.semanticscholar.org/CorpusID:279999326.

[37] Keyu Chen, Wenchao Sun, Hao Cheng, and Sifa Zheng. Rift: Group-relative rl fine-tuning for realistic and controllable traffic simulation, 2025. URL https://arxiv.org/abs/2505.03344.

[38] Muleilan Pei, Shaoshuai Shi, and Shaojie Shen. Advancing multi-agent traffic simulation via r1-style reinforcement fine-tuning. *ArXiv*, abs/2509.23993, 2025. URL https://api.semanticscholar.org/CorpusID:281675127.

[39] Yan Wang, Wenjie Luo, Junjie Bai, Yulong Cao, Tong Che, Ke Chen, Yuxiao Chen, Jenna Diamond, Yifan Ding, Wenhao Ding, et al. Alpamayo-r1: Bridging reasoning and action prediction for generalizable autonomous driving in the long tail. *arXiv preprint arXiv:2511.00088*, 2025.

[40] Wei Cao, Marcel Hallgarten, Tianyu Li, Daniel Dauner, Xunjiang Gu, Caojun Wang, Yakov Miron, Marco Aiello, Hongyang Li, Igor Gilitschenski, Boris Ivanovic, Marco Pavone, Andreas Geiger, and Kashyap Chitta. Pseudo-simulation for autonomous driving. In *Conference on Robot Learning (CoRL)*, 2025.

[41] Daniel Dauner, Marcel Hallgarten, Tianyu Li, Xinshuo Weng, Zhiyu Huang, Zetong Yang, Hongyang Li, Igor Gilitschenski, Boris Ivanovic, Marco Pavone, Andreas Geiger, and Kashyap Chitta. Navsim: Data-driven non-reactive autonomous vehicle simulation and benchmarking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[42] Songyou Peng, Kyle Genova, Chiyu "Max" Jiang, Andrea Tagliasacchi, Marc Pollefeys, and Thomas Funkhouser. Openscene: 3d scene understanding with open vocabularies, 2023. URL https://arxiv.org/abs/2211.15654.

[43] Holger Caesar, Juraj Kabzan, Kok Seang Tan, Whye Kit Fong, Eric Wolff, Alex Lang, Luke Fletcher, Oscar Beijbom, and Sammy Omari. Nuplan: A closed-loop ml-based planning benchmark for autonomous vehicles, 2022. URL https://arxiv.org/abs/2106.11810.

[44] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-vl technical report. *ArXiv*, abs/2502.13923, 2025. URL https://api.semanticscholar.org/CorpusID:276449796.

[45] Luke Rowe, Rodrigue de Schaetzen, Roger Girgis, Christopher Pal, and Liam Paull. Poutine: Vision-language-trajectory pre-training and reinforcement learning post-training enable robust end-to-end autonomous driving, 2025. URL https://arxiv.org/abs/2506.11234.

[46] Bernhard Jaeger, Daniel Dauner, Jens Beißwenger, Simon Gerstenecker, Kashyap Chitta, and Andreas Geiger. Carl: Learning scalable planning policies with simple rewards. *arXiv preprint arXiv:2504.17838*, 2025.

[47] Daniel Dauner, Marcel Hallgarten, Andreas Geiger, and Kashyap Chitta. Parting with misconceptions about learning-based vehicle motion planning. In *Conference on Robot Learning*, pages 1268–1281. PMLR, 2023.

[48] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

[49] Eitan Altman. *Constrained Markov decision processes*, volume 7. CRC Press, 1999.

[50] Martin Treiber, Ansgar Hennecke, and Dirk Helbing. Congested traffic states in empirical observations and microscopic simulations. *Physical review E*, 62(2):1805, 2000.

[51] Tianyu Li, Yihang Qiu, Zhenhua Wu, Carl Lindström, Peng Su, Matthias Nießner, and Hongyang Li. Mtgs: Multi-traversal gaussian splatting. *arXiv preprint arXiv:2503.12552*, 2025.

[52] Hidehisa Arai, Keita Miwa, Kento Sasaki, Yu Yamaguchi, Kohei Watanabe, Shunsuke Aoki, and Issei Yamamoto. Covla: Comprehensive vision-language-action dataset for autonomous driving. *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1933–1943, 2025. URL https://api.semanticscholar.org/CorpusID:271909725.

[53] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.

A Additional theorems and Proofs

Theorem A.1 (Theorem 4.1 restated)**.** *For a fixed sampled group, the sensitivity of the scalarized-reward GRPO estimator g^{ScRew} to the logit z_k of constraint k is*

$$\frac{\partial g^{\mathrm{ScRew}}}{\partial z_k} = \lambda_k \left(\frac{\partial g^{\mathrm{ScRew}}}{\partial \lambda_k} - \sum_{j \in \{R,1,\dots,K\}} \lambda_j \frac{\partial g^{\mathrm{ScRew}}}{\partial \lambda_j} \right), \tag{10}$$

where, letting $D := \lambda_R N_R - \sum_{k=1}^{K} \lambda_k N_k$, the partial sensitivities are

$$\frac{\partial g^{\mathrm{ScRew}}}{\partial \lambda_R} = \frac{N_R}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}} - \frac{D}{\hat{\sigma}^3_{S^{\boldsymbol{\lambda}}}} \widehat{\mathrm{Cov}}(S^{\boldsymbol{\lambda}}, R), \qquad \frac{\partial g^{\mathrm{ScRew}}}{\partial \lambda_k} = -\frac{N_k}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}} + \frac{D}{\hat{\sigma}^3_{S^{\boldsymbol{\lambda}}}} \widehat{\mathrm{Cov}}(S^{\boldsymbol{\lambda}}, C_k),$$

and $\widehat{\mathrm{Cov}}(S^{\boldsymbol{\lambda}}, X) := \frac{1}{G} \sum_{i=1}^{G} (S_i^{\boldsymbol{\lambda}} - \bar{S}^{\boldsymbol{\lambda}})(X_i - \bar{X})$ denotes the empirical group covariance.

Proof. For a fixed sampled group, define (as before)

$$g^{\mathrm{ScRew}}(\boldsymbol{\lambda}) = \frac{D}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}}, \qquad N_R := \frac{1}{G} \sum_{i=1}^{G} (R_i - \bar{R}) \psi_i, \qquad N_k := \frac{1}{G} \sum_{i=1}^{G} (C_{k,i} - \bar{C}_k) \psi_i.$$

Since the sampled group is fixed, the quantities N_R and $\{N_k\}_{k=1}^{K}$ do not depend on $\boldsymbol{\lambda}$. The dependence on $\boldsymbol{\lambda}$ enters through the linear numerator D and through the denominator $\hat{\sigma}_{S^{\boldsymbol{\lambda}}}$.

We first compute the partial sensitivities with respect to the multiplier coordinates. Recall that

$$S_i^{\boldsymbol{\lambda}} = \lambda_R R_i - \sum_{k=1}^{K} \lambda_k C_{k,i}, \qquad \bar{S}^{\boldsymbol{\lambda}} = \frac{1}{G} \sum_{i=1}^{G} S_i^{\boldsymbol{\lambda}}.$$

Derivative with respect to λ_k. For any constraint k,

$$\frac{\partial S_i^{\boldsymbol{\lambda}}}{\partial \lambda_k} = -C_{k,i}, \qquad \frac{\partial \bar{S}^{\boldsymbol{\lambda}}}{\partial \lambda_k} = -\bar{C}_k,$$

so

$$\frac{\partial (S_i^{\boldsymbol{\lambda}} - \bar{S}^{\boldsymbol{\lambda}})}{\partial \lambda_k} = -(C_{k,i} - \bar{C}_k).$$

Now write

$$\hat{\sigma}_{S^{\boldsymbol{\lambda}}} = \sqrt{\frac{1}{G} \sum_{i=1}^{G} (S_i^{\boldsymbol{\lambda}} - \bar{S}^{\boldsymbol{\lambda}})^2}\,.$$

Differentiating gives,

$$\begin{aligned} \frac{\partial \hat{\sigma}_{S^{\boldsymbol{\lambda}}}}{\partial \lambda_k} &= \frac{1}{2\hat{\sigma}_{S^{\boldsymbol{\lambda}}}} \frac{\partial}{\partial \lambda_k} \left(\frac{1}{G} \sum_{i=1}^{G} (S_i^{\boldsymbol{\lambda}} - \bar{S}^{\boldsymbol{\lambda}})^2 \right) \\ &= -\frac{1}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}} \frac{1}{G} \sum_{i=1}^{G} (S_i^{\boldsymbol{\lambda}} - \bar{S}^{\boldsymbol{\lambda}})(C_{k,i} - \bar{C}_k) \\ &= -\frac{\widehat{\mathrm{Cov}}(S^{\boldsymbol{\lambda}}, C_k)}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}}. \end{aligned} \tag{11}$$

Also,

$$\begin{aligned} \frac{\partial g^{\mathrm{ScRew}}}{\partial \lambda_k} &= \frac{1}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}} \frac{\partial D}{\partial \lambda_k} + D \frac{\partial}{\partial \lambda_k} \left(\frac{1}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}} \right) \\ &= -\frac{N_k}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}} - \frac{D}{\hat{\sigma}^2_{S^{\boldsymbol{\lambda}}}} \frac{\partial \hat{\sigma}_{S^{\boldsymbol{\lambda}}}}{\partial \lambda_k}. \end{aligned} \tag{12}$$

Substituting (11) into (12) gives

$$\frac{\partial g^{\mathrm{ScRew}}}{\partial \lambda_k} = -\frac{N_k}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}} + \frac{D}{\hat{\sigma}^3_{S^{\boldsymbol{\lambda}}}} \widehat{\mathrm{Cov}}(S^{\boldsymbol{\lambda}}, C_k). \tag{13}$$

Note that (13) shows the sensitivity of the policy gradient with respect to multipliers λ_i if these were not softmax-normalized.

The derivative with respect to λ_R follows a near identical derivation:

Derivative with respect to λ_R. We have

$$\frac{\partial (S^{\boldsymbol{\lambda}}_i - \bar{S}^{\boldsymbol{\lambda}})}{\partial \lambda_R} = R_i - \bar{R}.$$

Differentiating gives

$$\begin{aligned}
\frac{\partial \hat{\sigma}_{S^{\boldsymbol{\lambda}}}}{\partial \lambda_R} &= \frac{1}{2\hat{\sigma}_{S^{\boldsymbol{\lambda}}}} \frac{\partial}{\partial \lambda_R} \left(\frac{1}{G} \sum_{i=1}^{G} (S^{\boldsymbol{\lambda}}_i - \bar{S}^{\boldsymbol{\lambda}})^2 \right) \\
&= \frac{1}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}} \frac{1}{G} \sum_{i=1}^{G} (S^{\boldsymbol{\lambda}}_i - \bar{S}^{\boldsymbol{\lambda}})(R_i - \bar{R}) \\
&= \frac{\widehat{\mathrm{Cov}}(S^{\boldsymbol{\lambda}}, R)}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}}.
\end{aligned}$$

and so

$$\begin{aligned}
\frac{\partial g^{\mathrm{ScRew}}}{\partial \lambda_R} &= \frac{1}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}} \frac{\partial D}{\partial \lambda_R} + D \frac{\partial}{\partial \lambda_R} \left(\frac{1}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}} \right) \\
&= \frac{N_R}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}} - \frac{D}{\hat{\sigma}^2_{S^{\boldsymbol{\lambda}}}} \frac{\partial \hat{\sigma}_{S^{\boldsymbol{\lambda}}}}{\partial \lambda_R} \\
&= \frac{N_R}{\hat{\sigma}_{S^{\boldsymbol{\lambda}}}} - \frac{D}{\hat{\sigma}^3_{S^{\boldsymbol{\lambda}}}} \widehat{\mathrm{Cov}}(S^{\boldsymbol{\lambda}}, R).
\end{aligned} \tag{14}$$

Chain rule through the multiplier logits. Since the multipliers are softmax-normalized,

$$\lambda_j = \frac{\exp(z_j)}{\sum_{j' \in \{R,1,\ldots,K\}} \exp(z_{j'})},$$

their Jacobian is

$$\frac{\partial \lambda_j}{\partial z_k} = \lambda_j(\delta_{jk} - \lambda_k), \qquad j \in \{R, 1, \ldots, K\}.$$

Applying chain rule,

$$\begin{aligned}
\frac{\partial g^{\mathrm{ScRew}}}{\partial z_k} &= \sum_{j \in \{R,1,\ldots,K\}} \frac{\partial g^{\mathrm{ScRew}}}{\partial \lambda_j} \frac{\partial \lambda_j}{\partial z_k} \\
&= \sum_{j \in \{R,1,\ldots,K\}} \frac{\partial g^{\mathrm{ScRew}}}{\partial \lambda_j} \lambda_j(\delta_{jk} - \lambda_k) \\
&= \lambda_k \frac{\partial g^{\mathrm{ScRew}}}{\partial \lambda_k} - \lambda_k \sum_{j \in \{R,1,\ldots,K\}} \lambda_j \frac{\partial g^{\mathrm{ScRew}}}{\partial \lambda_j}.
\end{aligned} \tag{15}$$

Factoring out λ_k yields

$$\frac{\partial g^{\mathrm{ScRew}}}{\partial z_k} = \lambda_k \left(\frac{\partial g^{\mathrm{ScRew}}}{\partial \lambda_k} - \sum_{j \in \{R,1,\ldots,K\}} \lambda_j \frac{\partial g^{\mathrm{ScRew}}}{\partial \lambda_j} \right), \tag{16}$$

and combining (13), (14) with (16) proves the result. □

Corollary A.2 (Corollary 4.2 restated)**.** *For a fixed sampled group, the sensitivity of the scalarized-advantage GRPO estimator to the logit z_k of constraint k is*

$$\frac{\partial g^{\mathrm{ScAdv}}}{\partial z_k} = \lambda_k \left(-\frac{N_k}{\hat{\sigma}_{C_k}} - \lambda_R \frac{N_R}{\hat{\sigma}_R} + \sum_{k'=1}^{K} \lambda_{k'} \frac{N_{k'}}{\hat{\sigma}_{C_{k'}}} \right). \tag{17}$$

Proof. For a fixed sampled group, the scalarized-advantage GRPO estimator can be written as

$$g^{\mathrm{ScAdv}}(\boldsymbol{\lambda}) = \lambda_R \frac{N_R}{\hat{\sigma}_R} - \sum_{k=1}^{K} \lambda_k \frac{N_k}{\hat{\sigma}_{C_k}}.$$

Since the sampled group is fixed, the quantities N_R, N_k, $\hat{\sigma}_R$, and $\hat{\sigma}_{C_k}$ do not depend on $\boldsymbol{\lambda}$. Therefore, the partial sensitivities with respect to the multiplier coordinates are immediate:

$$\frac{\partial g^{\mathrm{ScAdv}}}{\partial \lambda_R} = \frac{N_R}{\hat{\sigma}_R}, \qquad \frac{\partial g^{\mathrm{ScAdv}}}{\partial \lambda_k} = -\frac{N_k}{\hat{\sigma}_{C_k}}. \tag{18}$$

Note that (18) shows the sensitivity of the policy gradient with respect to multipliers λ_i if these were not softmax-normalized. However, since

$$\lambda_j = \frac{\exp(z_j)}{\sum_{j' \in \{R,1,\ldots,K\}} \exp(z_{j'})},$$

Then combining (15) with (18), we get

$$\frac{\partial g^{\mathrm{ScAdv}}}{\partial z_k} = \lambda_k \left(-\frac{N_k}{\hat{\sigma}_{C_k}} - \lambda_R \frac{N_R}{\hat{\sigma}_R} + \sum_{k'=1}^{K} \lambda_{k'} \frac{N_{k'}}{\hat{\sigma}_{C_{k'}}} \right),$$

which proves the result. □

Theorem A.3 (Effective Multiplier for Scalarized Rewards)**.** *Let*

$$\mathbf{x} := \begin{bmatrix} R \\ C_1 \\ \vdots \\ C_K \end{bmatrix} \in \mathbb{R}^{K+1}, \qquad \boldsymbol{\lambda} := \begin{bmatrix} \lambda_R \\ -\lambda_1 \\ \vdots \\ -\lambda_K \end{bmatrix} \in \mathbb{R}^{K+1},$$

and define the scalarized reward $S^{\boldsymbol{\lambda}} := \boldsymbol{\lambda}^\top \mathbf{x}$*. Denote the within-group mean and covariance of* $\mathbf{x}$ *by* $\boldsymbol{\mu} := \mathbb{E}[\mathbf{x}]$ *and* $\boldsymbol{\Sigma} := \mathrm{Cov}(\mathbf{x})$*, and let* $\sigma_{S^{\boldsymbol{\lambda}}} := \sqrt{\mathrm{Var}(S^{\boldsymbol{\lambda}})} = \sqrt{\boldsymbol{\lambda}^\top \boldsymbol{\Sigma}\, \boldsymbol{\lambda}}$*. Further define per-component standardizations*

$$\mathbf{z} := \mathbf{D}^{-1}(\mathbf{x} - \boldsymbol{\mu}), \qquad \mathbf{D} := \mathrm{diag}(\sigma_R, \sigma_{C_1}, \ldots, \sigma_{C_K})$$

so that

$$\begin{aligned} Z_R &:= \frac{R - \mu_R}{\sigma_R}, \\ Z_{C_k} &:= \frac{C_k - \mu_{C_k}}{\sigma_{C_k}}, \quad k = 1, \ldots, K, \end{aligned} \tag{19}$$

where $\mu_{(\cdot)}$ *and* $\sigma_{(\cdot)}$ *denote the within-group mean and standard deviation of the corresponding component. Then the GRPO advantage obtained by standardizing the scalarized reward,*

$$A^{\mathrm{ScRew}} := \frac{S^{\boldsymbol{\lambda}} - \mathbb{E}[S^{\boldsymbol{\lambda}}]}{\sigma_{S^{\boldsymbol{\lambda}}}},$$

admits the expansion

$$A^{\mathrm{ScRew}} = \frac{(\mathbf{D}\boldsymbol{\lambda})^\top \mathbf{z}}{\sigma_{S^{\boldsymbol{\lambda}}}} = \sum_{j=0}^{K} \left(\frac{\lambda_j\, \sigma_j}{\sigma_{S^{\boldsymbol{\lambda}}}} \right) Z_j, \tag{20}$$

where we index $j = 0$ *as the main reward component (*$\lambda_0 \equiv \lambda_R$*,* $\sigma_0 \equiv \sigma_R$*) and* $j = k$ *corresponds to cost* C_k*. Consequently, the effective contribution of each standardized component* Z_j *is scaled by* $\lambda_j \sigma_j / \sigma_{S^{\boldsymbol{\lambda}}}$*, and therefore depends on within-group scales and (through* $\sigma_{S^{\boldsymbol{\lambda}}}$*) the full covariance structure* $\boldsymbol{\Sigma}$*.*

Proof. By definition, $S^{\boldsymbol{\lambda}} = \boldsymbol{\lambda}^\top \mathbf{x}$ and $\mathbb{E}[S^{\boldsymbol{\lambda}}] = \boldsymbol{\lambda}^\top \boldsymbol{\mu}$, hence

$$S^{\boldsymbol{\lambda}} - \mathbb{E}[S^{\boldsymbol{\lambda}}] = \boldsymbol{\lambda}^\top (\mathbf{x} - \boldsymbol{\mu}). \tag{21}$$

Moreover,

$$\mathrm{Var}(S^{\boldsymbol{\lambda}}) = \mathrm{Var}(\boldsymbol{\lambda}^\top \mathbf{x}) = \boldsymbol{\lambda}^\top \mathrm{Cov}(\mathbf{x})\, \boldsymbol{\lambda} = \boldsymbol{\lambda}^\top \boldsymbol{\Sigma}\, \boldsymbol{\lambda},$$

which yields $\sigma_{S^{\boldsymbol{\lambda}}} = \sqrt{\boldsymbol{\lambda}^\top \boldsymbol{\Sigma}\, \boldsymbol{\lambda}}$.

Substituting into (21) gives

$$S^{\boldsymbol{\lambda}} - \mathbb{E}[S^{\boldsymbol{\lambda}}] = \boldsymbol{\lambda}^\top \mathbf{D}\mathbf{z} = (\mathbf{D}\boldsymbol{\lambda})^\top \mathbf{z}.$$

Dividing both sides by $\sigma_{S^{\boldsymbol{\lambda}}}$ yields

$$A^{\mathrm{ScRew}} = \frac{S^{\boldsymbol{\lambda}} - \mathbb{E}[S^{\boldsymbol{\lambda}}]}{\sigma_{S^{\boldsymbol{\lambda}}}} = \frac{(\mathbf{D}\boldsymbol{\lambda})^\top \mathbf{z}}{\sigma_{S^{\boldsymbol{\lambda}}}},$$

which is the first equality in (20). Expanding the dot product gives the summation form

$$A^{\mathrm{ScRew}} = \sum_{j=0}^{K} \left(\frac{\lambda_j\, \sigma_j}{\sigma_{S^{\boldsymbol{\lambda}}}} \right) Z_j,$$

completing the proof. □

B Preliminaries

Markov Decision Processes (MDPs). A Markov decision process (MDP) is defined by $(\mathcal{S}, \mathcal{A}, P, R)$ [48], where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $P(\cdot \mid s, a)$ the transition kernel, and $R(s, a)$ the reward function. At time t, the agent samples an action $a_t \sim \pi(\cdot \mid s_t)$, transitions according to $s_{t+1} \sim P(\cdot \mid s_t, a_t)$, and receives reward $R(s_t, a_t)$. The trajectory distribution under π and the expected discounted return are

$$p_\pi(\tau) = P_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1} \mid s_t, a_t)\, \pi(a_t \mid s_t),$$
$$J_R(\pi) = \mathbb{E}_{\tau \sim p_\pi}\Big[\sum_{t=0}^{T-1} \gamma^t R(s_t, a_t) \Big],$$

where the objective is to learn a policy π^* that maximizes the expected discounted return, i.e. $\pi^* = \arg\max_\pi J_R(\pi)$.

Constrained Markov Decision Processes (CMDPs). A constrained MDP (CMDP) augments the MDP objective with K expected cost constraints [49]. Given cost functions $C_k : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ and thresholds d_k, define the feasible set of policies as

$$\Pi_C = \Big\{ \pi \in \Pi :\ J_{C_k}(\pi) \le d_k,\ \ k = 1, \dots, K \Big\},$$
$$J_{C_k}(\pi) = \mathbb{E}_{\tau \sim p_\pi}\Big[\sum_{t=0}^{T-1} \gamma^t C_k(s_t, a_t) \Big].$$

The constrained optimal policy maximizes return subject to feasibility:

$$\pi^* = \arg\max_{\pi \in \Pi}\ J_R(\pi) \quad \text{s.t.} \quad J_{C_k}(\pi) \le d_k,\ \ k = 1, \dots, K,$$

where stationary *stochastic* policies may be required for optimality [49].

C Gridworld Environment

C.1 Environment Details

Our gridworld is a 10×10 grid in which the agent is initialized uniformly at random and must navigate to a randomly sampled goal location. Reaching the goal yields a sparse terminal reward of 1.0. At the start of each episode, 20% of the tiles are designated as lava; stepping onto a lava tile incurs a penalty of 1.0 (i.e., a cost). The agent is also equipped with a battery that drains by 5% at every timestep and incurs a penalty whenever its charge falls below 10%. The action space consists of five discrete actions: moving in the four cardinal directions, or staying in place to recharge at a rate of 20% per timestep. The agent observes a compact state representation consisting of:

- its (x, y) position, normalized to $[0, 1]^2$;
- the goal (x, y) coordinates, normalized to $[0, 1]^2$;
- the current battery level in $[0, 1]$;
- a 5×5 local occupancy map centered on the agent, where entries indicate lava tiles (flattened to a 25-dimensional vector).

Episodes last at most 80 timesteps and terminate early only upon reaching the goal.

C.2 Implementation Details

We train a categorical policy parameterized by a two-layer MLP (hidden size 128 with ReLU activations) that outputs action logits. Policies are optimized with the GRPO clipped objective ($\epsilon = 0.2$) using 2 epochs per update, minibatches of size 2048, and an entropy bonus of 0.001. Training runs for 8000 update steps; each update collects 64 episodes via grouped rollouts (group

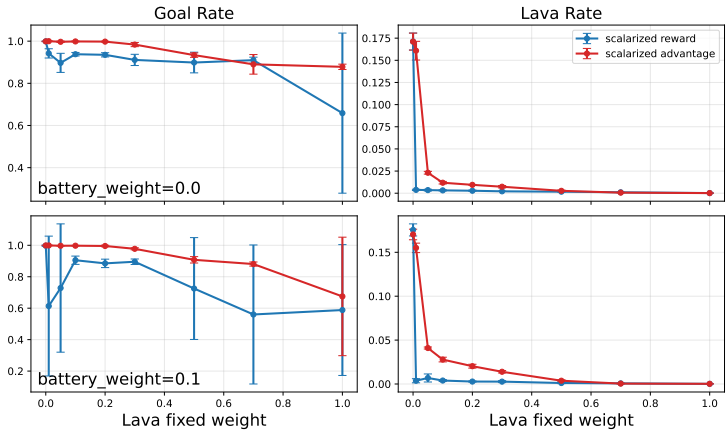

Figure 3: Final performance of GRPO policies evaluated over 1,000 episodes in the gridworld with fixed penalty weights, comparing *scalarized rewards* (blue) and *scalarized advantages* (red). Scalarized advantages consistently achieves higher goal-reaching rates while tracking the intended trade-off induced by the lava-weight sweep more faithfully, whereas scalarized rewards suppresses lava visitation disproportionately.

size 8 across 8 groups). We use Adam for the policy optimizer (learning rate 5×10^{-4} by default) and, when constraints are enabled, a separate Adam optimizer for cost weights (learning rate 10^{-2}). Cost weights are parameterized via a softmax over {reward, lava, battery} parameters and updated every step and initialized to 0.02 logits. All experiments were run on 16 CPU cores, and each run type required approximately 3 hours to complete across all five seeds.

C.3 Additional Results

We compare *scalarized rewards* and *scalarized advantages* under *fixed* reward and penalty weights to illustrate the benefit of scalarizing advantages in the unconstrained setting [7]. For simplicity, we restrict the battery-penalty weight to $\lambda_{\text{battery}}^{\text{fixed}} \in \{0.0, 0.1\}$ and sweep the lava-penalty weight over $\lambda_{\text{lava}}^{\text{fixed}} \in \{0.0, 0.01, 0.05, 0.1, 0.2, 0.3, 0.5, 0.7, 1.0\}$. Figure 3 illustrates how performance varies as we sweep $\lambda_{\text{lava}}^{\text{fixed}}$ for the two fixed settings of $\lambda_{\text{battery}}^{\text{fixed}}$. Overall, we observe consistently higher goal-reaching rates with *scalarized advantages*, along with more stable performance across seeds (through smaller error bars). Moreover, *scalarized rewards* tends to unilaterally optimize the lava rate: even the smallest non-zero lava weight, $\lambda_{\text{lava}}^{\text{fixed}} = 0.01$, leads to near-minimal lava visitation. In contrast, *scalarized advantages* provides smoother, more gradual control over the lava rate, which we argue better reflects the intended meaning of the weights. When multiple penalty terms are active ($\lambda_{\text{battery}}^{\text{fixed}} = 0.1$, i.e., bottom row in Fig. 3), *scalarized advantages* relaxes the lava rate for $\lambda_{\text{lava}}^{\text{fixed}} \leq 0.5$, whereas *scalarized rewards* behaves identically to the $\lambda_{\text{battery}}^{\text{fixed}} = 0.0$ setting, effectively optimizing the lava penalty in the same way regardless of the battery penalty.

Additionally, we show results for a single constraint in Figure 4, and for two constraints (battery and lava) in Figure 5. We refer the reader to the caption for more details.

D NAVSIM v2

D.1 Benchmark Details

In Stage 1, the benchmark evaluates closed-loop planning quality in an open-loop manner. Concretely, each planner outputs a 4-second trajectory sampled at 10 Hz, which is then executed forward without feedback using a kinematic bicycle model with an LQR controller. Unlike NAVSIM-v1 [41], which simply replayed recorded agent trajectories, NAVSIM-v2 introduces reactive behavior from other vehicles through the Intelligent Driver Model (IDM) [50], improving simulation realism. Stage 2 evaluates robustness under compounding error by synthesizing new driving rollouts using Multi-Traversal Gaussian Splatting [51]. These scenarios are initialized from perturbed ego states (e.g.,

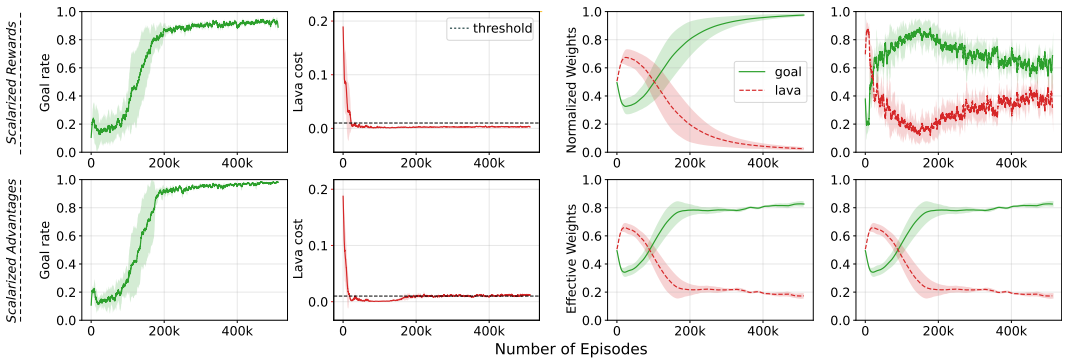

Figure 4: Learning dynamics on the gridworld under Constrained GRPO, comparing *scalarized advantages* (left) and *scalarized rewards* (right). In this experiment, we set $\tilde{d}_{\text{lava}} = 0.01$ and do not enforce the battery constraint. Scalarized advantages again produces a stronger final policy and makes effective use of the allowable cost budget, with the lava rate touching the specified threshold in order to allow for success on the main task. In contrast, scalarized rewards drives the constraint rate to near-zero early in training; despite small multipliers values, the corresponding effective weights remain larger and noisier, leading to less stable learning and lower final performance.

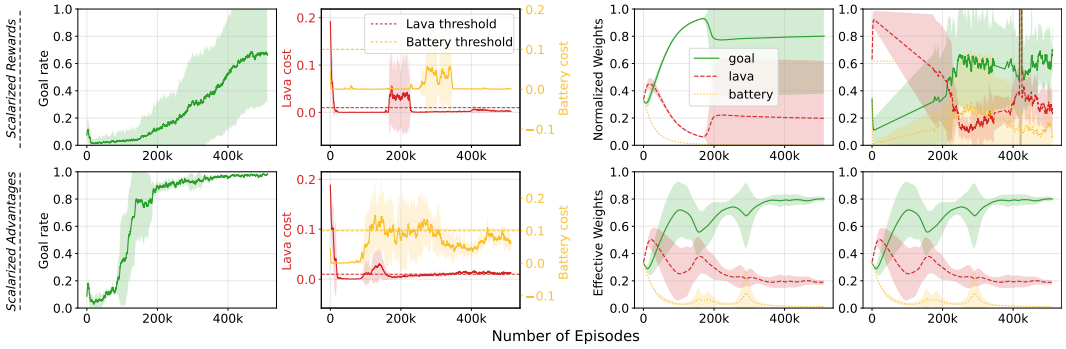

Figure 5: Learning dynamics on the gridworld under Constrained GRPO, comparing *scalarized advantages* (left) and *scalarized rewards* (right). In this experiment, we set $\tilde{d}_{\text{lava}} = 0.01$ and $\tilde{d}_{\text{battery}} = 0.1$. Scalarized advantages again produces a stronger final policy and makes effective use of its allowable costs' budget, with both the lava rate and battery rate stabilizing near their specified thresholds in order to allow for success on the main task. In contrast, scalarizing rewards drives both constraints' cost rate to near-zero early in training. Even with small values of multipliers, the corresponding effective weights remain larger and noisier, leading to less stable learning and lower final performance. Interestingly, we see that when it tries to creep higher, the effective multipliers react excessively, resulting in the agent not adequately navigating the environment, as shown by the poor goal success rate.

suboptimal position and/or heading), requiring the planner to produce corrective behavior. The resulting Navhard evaluation split contains 450 observations for Stage 1 and 5,462 observations for Stage 2.

The following is the expression for EPDMS:

$$\text{EPDMS} = \underbrace{\prod_{m \in \mathcal{M}_{\text{pen}}} \text{filter}_m(\text{agent}, \text{human})}_{\text{penalty terms}} \cdot \underbrace{\left(\frac{\sum_{m \in \mathcal{M}_{\text{avg}}} w_m \, \text{filter}_m(\text{agent}, \text{human})}{\sum_{m \in \mathcal{M}_{\text{avg}}} w_m} \right)}_{\text{weighted average terms}}. \quad (22)$$

To improve fairness, the function filter_m ignores penalty violations (i.e., returns 1 instead of 0) whenever the human expert driver makes the same violation. The weights w_m for the weighted average terms are: (1) $w_m = 5$ for EP, (2) $w_m = 5$ for TTC, (3) $w_m = 2$ for HC, (4) $w_m = 2$ for EC, and (5) $w_m = 2$ for LK.

D.2 Base Model Pretraining

We initialize all experiments from the Qwen2.5-VL 3B Instruct vision-language model [44]. We first perform supervised finetuning on the CoVLA dataset [52], which contains 83 hours of nominal driving collected in Japan. Following Rowe et al. [45], we represent planned trajectories as serialized text and train the model using next-token prediction in the original vocabulary.

To promote richer planning behavior, we additionally include automatically generated chain-of-thought annotations, produced in a zero-shot manner by Qwen2.5-VL 72B Instruct, and prepend these to the target trajectory sequences during training. After this initialization stage, we further supervise finetune the model on $92{,}000$ scenes from the NAVSIM training set [40, 41]. The remaining $\approx 11{,}000$ scenes are held out and used exclusively for reinforcement learning finetuning (RLFT) in our benchmark experiments.

D.3 Hyperparameters

We finetune all our models for two epochs through the 11k held-out scenes for RL finetuning (RLFT). All our experiments are performed on 8 A100 GPUs, with each RLFT run taking nearly 20 hours. We set $\beta = 0.02$ to constrain the policy to the reference SFT policy, the learning rate to $1e^{-6}$, and the group size to 16. We perform two iterations of policy optimization per batch of data. We update the Lagrangian multipliers at the same rate as the policy.

D.4 PPO and PPO Lagrangian Details

The value network is a separate Qwen2.5-VL 3B model initialized from the same SFT checkpoint as the policy. This backbone is finetuned using LoRA [53] and we attach separate linear value heads for each reward and constraint signal to the last hidden state of the Qwen model. Advantages are computed per-token using GAE ($\lambda = 0.95$, $\gamma = 0.99$), with the value network trained for 200 warmup steps before policy optimization begins.

D.5 Additional Results

In Figure 6, we show representative learning dynamics of the multipliers and corresponding constraints, highlighting differences between the two advantage estimation approaches for constrained GRPO in NAVSIM.

E GSM8K

E.1 Hyperparameters

We employ huggingface pretrained Qwen2.5-Instruct models with varying sizes. All models are RL-finetuned on a single node of 8x H200 GPUs, taking under three hours to complete four epochs of training, which was sufficient. We use the AdamW optimizer with a learning rate of $1e-6$ for 3B and 7B models, and a learning rate of $5e-6$ for 1.5B models; the Lagrangian parameters are optimized with the Adam optimizer with a learning rate of $1e-4$.

The constraint violation thresholds are different for different model sizes, since each has its own capability and setting thresholds too high can result in infeasible policies. For all model size, the format adherence and integer validity thresholds were set at 0.99; so the policy needed to respect the format and integer constraint 99% of the time. For the correctness constraint, the 1.5B sized model all employed a 75%; the 3B employed a 90% threshold; and the 7B employed a 95%. On the training set, these thresholds were all respected; however, as can be seen in Table 2, there is a small shift in the performance across the board (ppo-lagrangian and constrained GRPO).

F Limitations

In this section, we describe some of the limitations of constrained GRPO. In constrained policy optimization, a general problem is how one should choose the thresholds of the different behavioural constraints. While in this work we mitigate this problem by using interpretable indicator cost

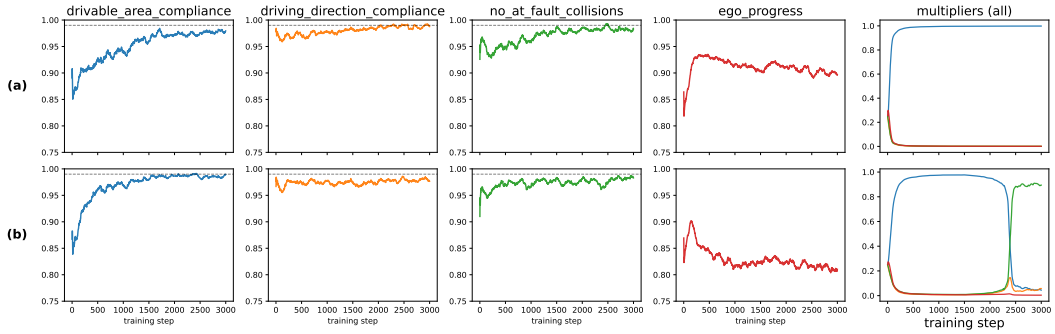

Figure 6: Constrained GRPO plots of constraint and multiplier values through training, comparing with **(a)** scalarized rewards with **(b)** scalarized advantages. Focusing on the ego progress and the drivable area compliance, we see that with the scalarized rewards, the policy does not adhere to the multiplier settings as we intend. First, the drivable area compliance (DAC) stays below its minimum threshold 0.99 throughout its training even though the multiplier value had moved close to its maximum value of 1.0. Moreover, we see that even though the ego-progress (a component of the WAT reward) is set to a very low value, the scalarized rewards variant still maximizes it. Conversely, the scalarized advantage model allows the different components to adhere to the multiplier setting. In addition, we see that at around 2500 training steps, something interesting happens in the scalarized advantage model's multipliers; since the DAC constraint is satisfied and the no at-fault collision (NC) takes a sharp dip, the Lagrangian optimization pushes the NC multiplier higher and squashes the DAC multiplier. This confirms our theoretical findings that scalarizing the advantages leads to better adherence to the specified multiplier weights.

functions, it is unclear how to effectively choose these thresholds such that the constraint remain feasible. In addition, with multiple constraints interacting and competing together, progress on certain constraints could stall completely when one of the constraints is infeasible, effectively killing the optimization process. An interesting direction for future work is to couple Constrained GRPO with curriculum-style mechanisms that adapt constraint thresholds over training, potentially improving exploration and enabling satisfaction of stricter constraints.